

内部汇报材料

# ModelNet 路由系统介绍

给第一次了解项目的同事看的非技术版说明

| 项目   | 说明                               |
|------|----------------------------------|
| 阅读对象 | 产品、项目、运营、测试、管理和非核心研发同事           |
| 阅读目标 | 听完能讲清它解决什么问题、怎么分流请求、为什么会自动选择不同模型 |
| 推荐讲法 | 先讲生活类比，再讲一次请求的旅程，最后讲系统价值和常见问题    |

### 一句话版本

ModelNet 路由系统像一个模型服务调度中心：它收到问题后，先判断需求，再从可用模型中选择最合适的一条路线；简单问题直接交给一个模型，复杂问题可以让多个模型合作。

## 1. 这套路由系统到底在做什么

可以把它理解成一个“分诊台”或“调度中心”。用户的问题不是直接丢给某一个固定模型，而是先进入 ModelNet 路由系统。系统会判断这个问题需要什么能力、当前哪些模型可用、哪些模型负载更低，然后决定把请求交给谁处理。

这样做的好处是：上层应用不用关心背后具体有多少模型，也不用自己判断哪个模型现在健康、哪个模型更适合。它只要把请求发给统一入口，路由系统负责把后面的选择做掉。

### 面向普通听众的类比

如果把每个模型看成一位专家，路由系统就是前台调度员。它先看问题类型和专家状态，再决定是请一位专家直接回答，还是组织几位专家一起给出更可靠的结论。

## 2. 为什么需要路由

- 模型很多，能力不同：有的适合中文，有的适合代码，有的支持结构化输出，有的响应更快。
- 模型状态会变化：有些后端可能正在重启、压力较高、暂时不可用，不能让用户直接踩到故障。
- 不同场景要求不同：简单问答追求速度，复杂分析更看重质量和稳定性。
- 上层应用需要统一入口：LobeHub 或 LiteLLM 只需要接一个稳定接口，不需要知道每个模型的内部地址。

## 3. 一次请求的旅程

一次用户请求大致会经过下面五步。这里不展开代码，只看业务上发生了什么。

| 步骤 | 发生了什么  | 通俗解释                                |
|----|--------|-------------------------------------|
| 1  | 收到问题   | 用户或应用发来一次提问。                        |
| 2  | 理解要求   | 系统判断要找什么能力，比如普通对话、结构化输出、工具调用或更高可靠性。 |
| 3  | 查看可用模型 | 只从当前可用、当前租户允许使用的模型里挑选。              |
| 4  | 选择路线   | 简单问题通常交给一个合适模型；复杂问题可能交给多个模型合作。      |
| 5  | 返回结果   | 把模型回答整理成客户端熟悉的格式，并附带必要的追踪信息。        |

## 4. 普通路由：选一个最合适的模型

普通路由适合大多数日常对话。系统会先筛掉不符合条件的模型，再从剩下的模型里挑一个当前更合适的后端。

筛选时主要看什么

- 权限：当前调用方是否允许使用这个模型。
- 能力：这个模型是否满足请求需要的能力。
- 可用性：模型服务是否在线，K8s 里是否有 ready 的实例。
- 负载：CPU、内存、GPU、显存和当前排队请求是否过高。
- 近期失败：如果某个后端刚刚失败过，会暂时降级或冷却，避免连续打到故障点。

核心原则

普通路由不是“随便找一个模型”，而是在满足权限和能力的前提下，优先选择当前更健康、更空闲、更可靠的后端。

5. 自动路由：复杂问题可以组织多模型协作

当请求使用 modelnet-auto 时，系统不会固定调用某一个模型，而是先判断任务复杂度。简单问题仍然走单模型，复杂问题才会升级为多个模型协作。

| 场景      | 系统倾向怎么做         | 对用户的意义       |
|---------|-----------------|--------------|
| 简单问答    | 选择一个当前最合适的模型    | 更快，资源消耗低     |
| 复杂分析    | 让多个模型分别给出答案，再综合 | 质量更稳，减少单模型偏差 |
| 需要更高可靠性 | 先拆解关键结论，再做验证    | 更适合需要谨慎回答的场景 |
| 系统负载很高  | 主动收缩为少量调用或单模型   | 保护整体服务稳定     |

可以把自动路由理解成一个“按需升级”的机制：它不是每次都开很多模型，而是根据问题难度、当前资源和质量目标决定要不要升级。

6. 路由系统带来的价值

| 对象  | 价值                    |
|-----|-----------------------|
| 对用户 | 入口更稳定，遇到后端波动时不容易直接失败。 |

|     |                                   |
|-----|-----------------------------------|
| 对产品 | 可以把多个模型包装成一个统一能力，而不是暴露复杂模型列表。     |
| 对运维 | 有健康检查、负载信息和错误冷却，问题更容易定位。          |
| 对研发 | 新增模型或策略时，可以放在统一网关里管理，不需要每个应用重复接入。 |

## 7. 怎么判断系统是否正常

不熟悉代码的同事可以从现象层面理解这几个检查点：

- 模型列表能正常返回，说明入口和权限配置基本可用。
- 能力列表能看到各模型支持什么，说明注册表和能力声明正常。
- 健康检查显示有可用后端，说明至少有模型服务可以被路由到。
- 一次测试请求能返回结果，并且能看到它实际选中的后端。
- 如果自动路由退回单模型，不一定是故障，可能是系统判断问题简单、资源紧张或预算不足。

## 8. 汇报时建议强调的三句话

1. 它解决的核心问题是：让上层应用不用直接面对一堆模型后端，而是面对一个统一、会判断状态的入口。
2. 它的普通路由是健康和负载感知的：不是只看模型名字，也会看当前系统状态。
3. 它的自动路由是按需协作的：简单问题追求快，复杂问题才组织多模型，提高回答质量和稳健性。

## 9. 少量必要名词

| 名词              | 通俗解释                           |
|-----------------|--------------------------------|
| ModelNet Router | 模型请求的统一入口和调度中心。                |
| LiteLLM         | 上层代理层，把应用请求转给 ModelNet Router。 |

|           |                               |
|-----------|-------------------------------|
| Candidate | 当前可以被选择的某个模型后端。               |
| 普通路由      | 从可用模型里选一个来回答。                 |
| 自动路由      | 根据问题难度和系统状态决定是否让多个模型协作。       |
| 健康状态      | 模型服务是否在线、是否有 ready 实例、近期是否失败。 |
| 负载        | CPU、内存、GPU、显存、排队请求等压力指标。      |

## 10. 最后一页总结

### 可以带走的结论

ModelNet 路由系统的价值，不是把请求简单转发出去，而是在模型很多、状态不断变化的情况下，帮上层应用选择一条更合适、更稳定的模型调用路线。

给普通同事汇报时，不需要展开每个函数。抓住三层就够了：统一入口、智能选择、稳定返回。理解这三层，就能理解这套路由系统为什么存在，以及它在 ModelNet ToC 里的作用。